Read the following code and explain what the code does and explain line by line of the code: % % Wheel torques: rear-drive + smooth ABS brake formulation
% Sign conventions in your model:
%   - Tt >= 0 : total rear drive torque request
%   - Tb <= 0 : total brake torque request
%   - SX < 0   : braking slip
%   - wheel dynamics remain: dot_omega = (Twheel - fx*veh.rw)/veh.Iw

kb = 0.65;   % exact front brake fraction

% ===== ABS tuning parameters =====
abs_trig   = 0.10;   % ABS trigger in braking-slip magnitude
abs_width   = 0.02;   % smooth activation width
abs_eta   = 0.70;   % global brake-relief strength, 0 <= abs_eta < 1
abs_lr_gain = 15.0;   % left-right redistribution sensitivity
abs_eps_pos = 1e-4;   % smooth positive-part regularization
abs_eps_max = 1e-4;   % smooth-max regularization

% ===== Smooth braking-slip magnitudes from your SX definition =====
% Since SX is negative under braking, use smooth max(-SX, 0)
absSlip_fl = 0.5*((-SXfl) + sqrt(SXfl^2 + abs_eps_pos^2));
absSlip_fr = 0.5*((-SXfr) + sqrt(SXfr^2 + abs_eps_pos^2));
absSlip_rl = 0.5*((-SXrl) + sqrt(SXrl^2 + abs_eps_pos^2));
absSlip_rr = 0.5*((-SXrr) + sqrt(SXrr^2 + abs_eps_pos^2));

% ===== Smooth maxima =====
absSlip_f = 0.5*(absSlip_fl + absSlip_fr + sqrt((absSlip_fl - absSlip_fr)^2 + abs_eps_max^2));
absSlip_r = 0.5*(absSlip_rl + absSlip_rr + sqrt((absSlip_rl - absSlip_rr)^2 + abs_eps_max^2));
absSlip_g = 0.5*(absSlip_f + absSlip_r + sqrt((absSlip_f - absSlip_r)^2 + abs_eps_max^2));

% ===== Smooth ABS gates =====
g_f   = 0.5*(1 + tanh((absSlip_f - abs_trig)/abs_width));
g_r   = 0.5*(1 + tanh((absSlip_r - abs_trig)/abs_width));
g_abs = 0.5*(1 + tanh((absSlip_g - abs_trig)/abs_width));

% ===== Global brake relief while preserving exact front-rear ratio =====
% Tb <= 0, so multiplying by (1 - abs_eta*g_abs) makes braking less negative
Tb_eff = (1 - abs_eta*g_abs) * Tb;

% ===== Exact front-rear brake split =====
Tbf = kb * Tb_eff;   % total front brake torque <= 0
Tbr = (1-kb) * Tb_eff;   % total rear brake torque <= 0

% ===== Smooth left-right redistribution inside each axle =====
% If left wheel has larger braking-slip magnitude than right,
% Delta > 0 and the left brake torque becomes less negative.
Delta_f = -0.5 * Tbf * g_f * tanh(abs_lr_gain * (absSlip_fl - absSlip_fr));
Delta_r = -0.5 * Tbr * g_r * tanh(abs_lr_gain * (absSlip_rl - absSlip_rr));

% ===== Front wheel torques (front wheels only brake) =====
Tbfl = 0.5*Tbf + Delta_f;
Tbfr = 0.5*Tbf - Delta_f;

Tfl = Tbfl;
Tfr = Tbfr;

% ===== Rear wheel torques = equal drive split + rear brake split =====
Tbrl = 0.5*Tbr + Delta_r;
Tbrr = 0.5*Tbr - Delta_r;

Trl = 0.5*Tt + Tbrl;
Trr = 0.5*Tt + Tbrr;

Thought for 14s

This code builds a **rear-wheel-drive wheel-torque allocation law with a smooth ABS-style brake intervention**.

At a high level, it does four things:

1. It reads the four wheel slip ratios and extracts the **braking slip magnitudes** smoothly.
2. It detects whether any wheel or axle is entering excessive braking slip, using **smooth gates** instead of hard on/off logic.
3. It reduces the total brake request smoothly when ABS is active, while keeping the **front-rear brake ratio exactly fixed**.
4. Inside each axle, it redistributes brake torque **left-to-right** so the more-slipping wheel gets relieved and the less-slipping wheel gets more braking.

The rear wheels also receive the drive torque request, so the final rear wheel torques are:

- rear drive split equally left/right
- plus rear brake torque, possibly ABS-modified and left-right redistributed

---

What the whole code is doing physically

Your sign convention is:

- `Tt >= 0` : total rear drive torque request
- `Tb <= 0` : total brake torque request
- `SX < 0` : braking slip
- wheel rotational dynamics:

$$\dot{\omega} = \frac{T_{\text{wheel}} - f_x r_w}{I_w}$$

So under braking, the slip ratio becomes more negative.
This code therefore uses `-SX` as the raw braking-slip quantity.

The ABS behavior here is **smooth**, not hard switching:

- when braking slip is small, the code does almost nothing
- when braking slip approaches the threshold, the code smoothly activates
- when braking slip gets too large, the code:
  - reduces the total brake demand
  - redistributes brake side-to-side on each axle

This is designed to be more NLP-friendly than hard `if/else`, `max`, or discontinuous ABS logic.

Line-by-line explanation

Header comments

```
% % Wheel torques: rear-drive + smooth ABS brake formulation
% Sign conventions in your model:
%   - Tt >= 0  : total rear drive torque request
%   - Tb <= 0  : total brake torque request
%   - SX < 0   : braking slip
%   - wheel dynamics remain: dot_omega = (Twheel - fx*veh.rw)/veh.Iw
```

This is just describing the purpose and the sign convention.

- The code computes wheel torques.
- Rear axle is driven.
- Braking includes a smooth ABS mechanism.
- The wheel dynamics are assumed elsewhere, and these torques will be inserted into that model.

Important point: because `Tb <= 0`, **more negative means more braking torque**.

---

1. Fixed front-rear brake split

```
kb = 0.65;   % exact front brake fraction
```

MATLAB

This defines the front brake bias.

- `kb = 0.65` means **65% of total brake torque goes to the front axle**
- the remaining `35%` goes to the rear axle

So later:

- front axle gets `0.65 * Tb_eff`
- rear axle gets `0.35 * Tb_eff`

Because `Tb_eff <= 0`, both are braking torques.

---

2. ABS tuning parameters

```
% ===== ABS tuning parameters =====
abs_trig    = 0.10;   % ABS trigger in braking-slip magnitude
abs_width   = 0.02;   % smooth activation width
abs_eta     = 0.70;   % global brake-relief strength, 0 <= abs_eta < 1
abs_lr_gain = 15.0;   % left-right redistribution sensitivity
abs_eps_pos = 1e-4;   % smooth positive-part regularization
abs_eps_max = 1e-4;   % smooth-max regularization
```

These are tuning constants.

`abs_trig = 0.10`

This is the braking-slip threshold where ABS begins to activate.

Because your braking slip is measured by `absSlip = smooth max(-SX,0)`, ABS starts activating when braking slip magnitude is around `0.10`.

`abs_width = 0.02`

This controls how sharply ABS turns on.

It is used inside `tanh((x - abs_trig)/abs_width)`.

- smaller `abs_width` = more abrupt activation
- larger `abs_width` = smoother, more gradual activation

`abs_eta = 0.70`

This is the strength of the **global brake relief**.

Later the effective brake request is

MATLAB

So when ABS is fully active (`g_abs ≈ 1`),

$$Tb_{\text{eff}} \approx (1 - 0.70)Tb = 0.30Tb$$

Since `Tb` is negative, multiplying by `0.30` makes it **less negative**, meaning **much less braking**.

`abs_lr_gain = 15.0`

This controls how sensitively the code shifts braking torque left-right within each axle.

It is used in:

MATLAB

So:

- large value = quicker/stronger redistribution when left/right slip differs
- smaller value = gentler redistribution

`abs_eps_pos = 1e-4`

This regularizes the smooth positive-part function.

Instead of using hard:

$$\max(-SX, 0)$$

the code uses a smooth approximation involving a square root.
This avoids a non-differentiable kink at zero.

`abs_eps_max = 1e-4`

This regularizes the smooth maximum function.

Instead of hard:

$$\max(a, b)$$

the code uses a smooth approximation so derivatives remain continuous.

---

3. Smooth braking-slip magnitudes

```
% ===== Smooth braking-slip magnitudes from your SX definition =====
% Since SX is negative under braking, use smooth max(-SX, 0)
absSlip_fl = 0.5*((-SXfl) + sqrt(SXfl^2 + abs_eps_pos^2));
absSlip_fr = 0.5*((-SXfr) + sqrt(SXfr^2 + abs_eps_pos^2));
absSlip_rl = 0.5*((-SXrl) + sqrt(SXrl^2 + abs_eps_pos^2));
absSlip_rr = 0.5*((-SXrr) + sqrt(SXrr^2 + abs_eps_pos^2));
```

These four lines compute the **braking-slip magnitude** at each wheel:

- `absSlip_fl` : front-left
- `absSlip_fr` : front-right
- `absSlip_rl` : rear-left
- `absSlip_rr` : rear-right

Because your braking slip is negative, the natural braking magnitude would be:

$$\max(-SX, 0)$$

This code uses the smooth approximation:

$$\text{smoothpos}(x) = \frac{1}{2}\left(x + \sqrt{x^2 + \varepsilon^2}\right)$$

with $x = -SX$.

So:

- if `SX` is negative and large in magnitude, then `-SX > 0` , so `absSlip ≈ -SX`
- if `SX` is positive or near zero, then `absSlip = 0`

Why this matters

ABS should only react to **braking slip**, not drive slip.
So this extracts only the braking part of slip, smoothly.

---

4. Smooth maxima across wheels and axles

```
% ===== Smooth maxima =====
absSlip_f = 0.5*(absSlip_fl + absSlip_fr + sqrt((absSlip_fl - absSlip_fr)^2 + abs_eps_max^2));
absSlip_r = 0.5*(absSlip_rl + absSlip_rr + sqrt((absSlip_rl - absSlip_rr)^2 + abs_eps_max^2));
```

MATLAB

These lines compute smooth maximum slip values.

`absSlip_f`

This is the smooth maximum of the two front-wheel braking slips:

$$absSlip_f \approx \max(absSlip_{fl}, absSlip_{fr})$$

So it represents the worst braking slip on the **front axle**.

`absSlip_r`

This is the smooth maximum of the two rear-wheel braking slips:

$$absSlip_r \approx \max(absSlip_{rl}, absSlip_{rr})$$

So it represents the worst braking slip on the **rear axle**.

`absSlip_g`

This is the smooth maximum of the front and rear axle maxima:

$$absSlip_g \approx \max(absSlip_f, absSlip_r)$$

So it represents the **worst braking slip anywhere on the vehicle**.

Why use smooth maxima?

Hard `max()` is non-smooth at equality.
This smooth version keeps the function differentiable and better suited for gradient-based solvers.

---

5. Smooth ABS activation gates

```
% ===== Smooth ABS gates =====
g_f   = 0.5*(1 + tanh((absSlip_f - abs_trig)/abs_width));
g_r   = 0.5*(1 + tanh((absSlip_r - abs_trig)/abs_width));
```

MATLAB

These three lines convert slip magnitudes into smooth activation signals between 0 and 1.

General form:

$$g(x) = \frac{1}{2}\left(1 + \tanh\left(\frac{x - x_{\text{trig}}}{w}\right)\right)$$

Behavior:

- if `x << trigger` , then `g ≈ 0`
- if `x >> trigger` , then `g ≈ 1`
- near the trigger, `g` transitions smoothly

`g_f`

Front axle ABS activity.

- near 0 when front axle braking slip is small
- near 1 when front axle braking slip is excessive

`g_r`

Rear axle ABS activity.

- near 0 when rear axle braking slip is small
- near 1 when rear axle braking slip is excessive

`g_abs`

Global ABS activity for the whole vehicle.

This depends on `absSlip_g` , the worst slip anywhere on the car.

So if any wheel gets into large braking slip, `g_abs` will rise.

---

6. Global brake relief

```
% ===== Global brake relief while preserving exact front-rear ratio =====
% Tb <= 0, so multiplying by (1 - abs_eta*g_abs) makes braking less negative
```
MATLAB

This computes the **effective brake request** after ABS intervention.

Because:

- `Tb <= 0`
- `0 <= g_abs <= 1`
- `0 <= abs_eta < 1`

the factor `(1 - abs_eta*g_abs)` lies between `1` and `1-abs_eta`.

So as ABS activates:

- `Tb_eff` becomes closer to zero
- meaning less braking torque is applied

Example

If:

- `Tb = -1000`
- `g_abs = 1`
- `abs_eta = 0.70`

then:

$$Tb_{\text{eff}} = (1 - 0.70)(-1000) = -300$$

So the brake request is reduced from `-1000` to `-300`.

Important physical role

This is the main ABS-style brake pressure relief mechanism.

---

7. Exact front-rear brake split

```
% ===== Exact front-rear brake split =====
Tbf = kb * Tb_eff;          % total front brake torque <= 0
```
MATLAB

This splits the effective total brake torque into front and rear portions.

- `Tbf` = total front axle brake torque
- `Tbr` = total rear axle brake torque

Because `kb = 0.65`, this means:

- front gets 65%
- rear gets 35%

And because the split is applied **after** the global ABS relief, the front-rear ratio remains exact even when ABS is active.

That is an important design feature of this code.

---

8. Left-right redistribution within each axle

```
% ===== Smooth left-right redistribution inside each axle =====
% If left wheel has larger braking-slip magnitude than right,
% Delta > 0 and the left brake torque becomes less negative.
Delta_f = -0.5 * Tbf * g_f * tanh(abs_lr_gain * (absSlip_fl - absSlip_fr));
Delta_r = -0.5 * Tbr * g_r * tanh(abs_lr_gain * (absSlip_rl - absSlip_rr));
```

These are very important lines.

They create a left-right torque difference on each axle so that the more-slipping wheel gets relieved.

Structure of the formula

Take the front axle:

$$\Delta_f = -0.5\, T_{bf}\, g_f\, \tanh(k(absSlip_{fl} - absSlip_{fr}))$$

Now note:

- `Tbf <= 0`
- therefore `-0.5*Tbf >= 0`
- `g_f >= 0`
- `tanh(...)` is between `-1` and `1`

So `Delta_f` can be positive or negative depending on which wheel slips more.

Case 1: left front slips more than right front

Then:

$$absSlip_{fl} > absSlip_{fr}$$

so

$$\tanh(\cdot) > 0$$

therefore

$$\Delta_f > 0$$

Later:

```
Tbfl = 0.5*Tbf + Delta_f;
```
MATLAB

Since `0.5*Tbf` is negative, adding a positive `Delta_f` makes the left brake torque **less negative**, so the left wheel is relieved.

At the same time, the right wheel becomes **more negative**, so it gets more brake torque.

So brake torque is shifted away from the more-slipping wheel.

Case 2: right front slips more than left front

Then `Delta_f < 0`.

That gives:

- left brake becomes more negative
- right brake becomes less negative

Again, the more-slipping wheel is relieved.

Same logic for rear axle

`Delta_r` does the same thing for the rear axle.

Why multiply by `g_f` or `g_r`?

This means redistribution is weak or zero when ABS is inactive, and only becomes significant when axle slip is high.

So the code does not aggressively redistribute at normal low-slip conditions.

---

9. Front wheel torques

```
% ===== Front wheel torques (front wheels only brake) =====
Tbfl = 0.5*Tbf + Delta_f;
```
MATLAB

This starts from equal split of front-axle brake torque:

- each wheel would get `0.5*Tbf`

Then `Delta_f` shifts some torque from one side to the other.

So:

- `Tbfl` : front-left brake torque
- `Tbfr` : front-right brake torque

The sum is preserved:

$$Tbfl + Tbfr = Tbf$$

So redistribution does not change total front axle braking, only the side-to-side distribution.

```
Tfl = Tbfl;
```
MATLAB

These set the total front wheel torques.

Because the front axle is not driven in this model, front wheel torque equals brake torque only.

So:

- `Tfl` = total torque at front-left wheel
- `Tfr` = total torque at front-right wheel

Both are typically non-positive under braking.

---

10. Rear brake torques

```
% ===== Rear wheel torques = equal drive split + rear brake split =====
Tbrl = 0.5*Tbr + Delta_r;
```
MATLAB

This does the same left-right brake split for the rear axle.

- `Tbrl` : rear-left brake torque
- `Tbrr` : rear-right brake torque

Again:

$$Tbrl + Tbrr = Tbr$$

So total rear braking is preserved, but redistributed side-to-side.

---

11. Final rear wheel torques

```
Trl = 0.5*Tt + Tbrl;
```
MATLAB

These are the final torques applied to the rear wheels.

The drive torque request `Tt` is split equally:

- left rear gets `0.5*Tt`
- right rear gets `0.5*Tt`

Then rear brake torque is added:

- `Trl = drive part + brake part`
- `Trr = drive part + brake part`

Since `Tbrl` and `Tbrr` are negative, they subtract from drive torque.

So depending on the values:

- under pure driving: `Tb = 0`, so rear torques are just `0.5*Tt`
- under pure braking: `Tt = 0`, so rear torques are just rear brake torques
- under overlap: both contribute algebraically

Compact summary of each variable

Inputs / assumed existing variables

- `Tt` : total rear drive torque request, nonnegative
- `Tb` : total brake torque request, nonpositive
- `SXfl, SXfr, SXrl, SXrr` : slip ratios at the four wheels, negative under braking

Intermediate ABS quantities

- `absSlip_*` : smooth braking-slip magnitudes
- `absSlip_f` : front axle worst braking slip
- `absSlip_r` : rear axle worst braking slip
- `absSlip_g` : global worst braking slip
- `g_f, g_r, g_abs` : smooth activation levels in `[0,1]`

Brake torque quantities

- `Tb_eff` : globally ABS-relieved brake request
- `Tbf, Tbr` : front and rear axle brake torques
- `Delta_f, Delta_r` : left-right redistribution offsets
- `Tbfl, Tbfr, Tbrl, Tbrr` : per-wheel brake torques

Final wheel torques

- `Tfl, Tfr` : front wheel torques
- `Trl, Trr` : rear wheel torques

What is elegant about this formulation

This code has several nice properties.

1. Smooth everywhere

It avoids hard:

- `max`
- `if/else`
- sign switches
- discontinuous ABS toggles

That is much friendlier for gradient-based optimization and smooth simulation.

---

2. Correct sign behavior with your convention

Because braking slip is negative in your model, it correctly uses:

$$\max(-SX, 0)$$

in smooth form.

---

3. Exact front-rear brake ratio preserved

The global ABS relief is applied before front-rear splitting, so the chosen brake bias `kb` stays exact.

---

4. Left-right relief works in the right direction

The more-slipping wheel gets less braking torque, smoothly.

---

5. Axle total braking is preserved during redistribution

Within each axle, `+Delta` on one side and `-Delta` on the other means the axle total remains unchanged.

One important modeling note

This code allows:

```
Trl = 0.5*Tt + Tbrl
```
MATLAB

So if upstream logic ever gives both:

- `Tt > 0`
- `Tb < 0`

at the same time, the rear wheel torques become the algebraic sum of drive and brake.

That may be intentional for a smooth optimization formulation, but physically a real vehicle would usually have supervisory logic preventing strong simultaneous drive and brake requests.

So this code is a **smooth torque composition law**, not a full supervisory brake/drive arbitration logic.

In one sentence

This code computes four wheel torques for a rear-wheel-drive car by taking the requested rear drive torque and brake torque, then applying a smooth ABS mechanism that detects excessive braking slip, reduces total braking smoothly, and redistributes brake torque left-right so the more-slipping wheel is relieved.

If you want, next I can rewrite the whole block with **inline comments on every single line**, so it reads like an annotated version of your code.

Double check whether your explanation was fully correct.

I'm re-checking the sign logic, the smooth-max formulas, and whether any of my earlier wording overstated something.

Thought for 39s

Mostly yes. My explanation was **substantively correct**, but a few places need tighter wording.

What I stand by as correct

These parts were correct:

- `Tb <= 0`, so making `Tb` closer to zero means **less braking**.
- `absSlip_*` is a smooth version of `max(-SX,0)`, which is appropriate because your braking slip is negative.
- `absSlip_f`, `absSlip_r`, `absSlip_g` are smooth approximations of the front, rear, and global maxima.
- `g_f`, `g_r`, `g_abs` are smooth gates in `(0,1)`.
- `Tb_eff = (1 - abs_eta*g_abs) * Tb` gives smooth global brake relief.
- The **total** front-rear brake split is preserved exactly after global relief:
  - `Tbf = kb*Tb_eff`
  - `Tbr = (1-kb)*Tb_eff`
- `Delta_f` and `Delta_r` do shift brake torque away from the more-slipping wheel.
- Front axle total brake torque is preserved:
  - `Tbfl + Tbfr = Tbf`
- Rear axle total brake torque is preserved:
  - `Tbrl + Tbrr = Tbr`
- Final rear torques are equal drive split plus redistributed rear braking:
  - `Trl = 0.5*Tt + Tbrl`
  - `Trr = 0.5*Tt + Tbrr`

What needed more precise wording

1. "Smooth max" and "smooth positive part" are only approximate

Earlier I described them as if they were the exact positive part or exact maximum. More precisely:

- `absSlip_fl = 0.5*((-SXfl) + sqrt(SXfl^2 + abs_eps_pos^2))`
  is a **smooth approximation** of `max(-SXfl,0)`, not exactly equal to it.
- Likewise the "smooth maxima" are **approximations** of `max(a,b)`.

So when I said things like "this equals the braking-slip magnitude," the precise statement should be:

- it gives a **regularized smooth approximation** of braking-slip magnitude.

That matters mathematically, because with `abs_eps_pos > 0` and `abs_eps_max > 0`, these quantities are never perfectly sharp.

---

2. The gates are not exactly zero or one

I said things like "near 0" and "near 1," which was fine. But to be exact:

- because of the `tanh`, `g_f`, `g_r`, `g_abs` are strictly between 0 and 1 for finite inputs

- they only approach 0 or 1 asymptotically

So the ABS is never literally hard-off or hard-on.

---

3. "If any wheel slips too much, `g_abs` rises" is physically right, but mathematically approximate

That statement is fine in engineering language, but more precisely:

- `g_abs` depends on `absSlip_g`
- `absSlip_g` is a smooth approximation of the global maximum braking slip
- so `g_abs` is dominated by the worst-slip wheel, but not via an exact hard max

So the earlier explanation was correct in intent, but slightly informal.

---

4. The exact front-rear ratio is preserved for axle totals, not for individual wheels

My earlier wording was okay, but the precise statement is:

- the **axle-level** front/rear brake split is exact:

$$\frac{Tbf}{Tbr} = \frac{kb}{1-kb}$$

  whenever `Tb_eff ~= 0`
- after that, left-right redistribution changes the **per-wheel** torques inside each axle

So "exact front-rear brake split" refers to **axle totals**, not each wheel.

One extra property I should have mentioned

This code also guarantees that left-right redistribution cannot reverse brake sign on a wheel.

Because:

- `|tanh(...)| <= 1`
- `0 <= g_f, g_r <= 1`

we get:

- `|Delta_f| <= -0.5*Tbf`
- `|Delta_r| <= -0.5*Tbr`

since `Tbf <= 0`, `Tbr <= 0`.

That means:

- `Tbfl, Tbfr, Tbrl, Tbrr` remain `<= 0`

So redistribution can reduce a wheel's braking toward zero, but not turn it into a positive drive torque by itself.

That is a good property.

Final verdict

My earlier explanation was **correct in the main logic, sign interpretation, and physical meaning**.

The only things that needed tightening were:

- "equals" should really be "smooth approximation of"
- "turns on/off" should really be "smoothly approaches on/off"
- "exact front-rear split" should be understood as **axle-total split**

So I would say the explanation was **mostly fully correct physically and mathematically**, but it benefited from these precision fixes.

If you want, I can now give you a **second-pass corrected line-by-line explanation** with the wording cleaned up to be mathematically exact.

Yes please give me a second-pass corrected line-by-line explanation with the wording cleaned up to be mathematically exact.

I'm rewriting it more carefully so every statement matches the math exactly, especially the regularized max functions and the sign logic.

Thought for a few seconds

Below is a **second-pass, mathematically precise** line-by-line explanation.

I will be careful to distinguish:

- **exact equalities** from **smooth approximations**
- **axle-total quantities** from **per-wheel quantities**
- **sign conventions** from **physical interpretation**

---

1. Header comments

```
% % Wheel torques: rear-drive + smooth ABS brake formulation
% Sign conventions in your model:
%   - Tt >= 0  : total rear drive torque request
%   - Tb <= 0  : total brake torque request
%   - SX < 0   : braking slip
%   - wheel dynamics remain: dot_omega = (Twheel - fx*veh.rw)/veh.Iw
```

These comments state the intended interpretation of the variables used in the block.

Exact meaning

- `Tt >= 0` means the commanded **total rear-axle drive torque** is nonnegative.
- `Tb <= 0` means the commanded **total vehicle brake torque** is nonpositive.
- `SX < 0` under braking means your slip-ratio definition is such that **braking corresponds to negative slip ratio**.
- The wheel rotational dynamics are assumed to be

$$\dot{\omega} = \frac{T_{\text{wheel}} - f_x\, r_w}{I_w}.$$

So this block is computing the wheel torque inputs:

- `Tfl, Tfr, Trl, Trr`

to be used in those wheel dynamics.

---

2. Front brake fraction

MATLAB

This defines a constant scalar front brake fraction.

Exact mathematical role

Later the axle-total effective brake torques are defined as

$$T_{bf} = k_b\, T_{b,\text{eff}}, \qquad T_{br} = (1-k_b)\, T_{b,\text{eff}}.$$

So with `kb = 0.65`:

- the **front axle total** gets 65% of the effective brake torque
- the **rear axle total** gets 35% of the effective brake torque

This split is exact at the **axle-total level**.

---

3. ABS tuning parameters

```
% ===== ABS tuning parameters =====
abs_trig    = 0.10;   % ABS trigger in braking-slip magnitude
abs_width   = 0.02;   % smooth activation width
abs_eta     = 0.70;   % global brake-relief strength, 0 <= abs_eta < 1
abs_lr_gain = 15.0;   % left-right redistribution sensitivity
abs_eps_pos = 1e-4;   % smooth positive-part regularization
abs_eps_max = 1e-4;   % smooth-max regularization
```

These are constants that shape the smooth ABS behavior.

`abs_trig = 0.10`

This is the slip-magnitude threshold around which the smooth ABS gates transition.

It is not a hard switching threshold.
Rather, it is the center of a smooth transition in the `tanh` gate.

`abs_width = 0.02`

This controls the width of the gate transition.

The gate has the form

$$g(x) = \frac{1}{2}\left(1 + \tanh\left(\frac{x - \text{abs\_trig}}{\text{abs\_width}}\right)\right).$$

So `abs_width` controls how rapidly the gate moves from values near 0 to values near 1 as `x` crosses `abs_trig`.

- smaller `abs_width` → sharper transition
- larger `abs_width` → smoother, broader transition

`abs_eta = 0.70`

This controls the maximum strength of global brake relief.

Later:

$$T_{b,\text{eff}} = (1 - \text{abs\_eta}\, g_{\text{abs}})\, T_b.$$

Since `0 <= g_abs < 1` for finite inputs and `0 <= abs_eta < 1`, the multiplier lies in the interval

$$1 - \text{abs\_eta} < (1 - \text{abs\_eta}\, g_{\text{abs}}) \le 1.$$

Because `Tb <= 0`, multiplying by a factor in `(1-abs_eta, 1]` makes the result:

- still nonpositive
- but generally **closer to zero**, hence **less braking**

`abs_lr_gain = 15.0`

This sets the sensitivity of the left-right redistribution to slip imbalance.

It appears inside

$$\tanh\left(\text{abs\_lr\_gain} \cdot (\text{left slip} - \text{right slip})\right).$$

So it controls how quickly redistribution saturates as left-right slip difference grows.

`abs_eps_pos = 1e-4`

This regularizes the smooth approximation of the positive-part function.

Instead of using the nonsmooth function

$$\max(x, 0),$$

the code uses

$$\phi_\varepsilon(x) = \frac{1}{2}\left(x + \sqrt{x^2 + \varepsilon^2}\right).$$

Here `abs_eps_pos` is that ε.

`abs_eps_max = 1e-4`

This regularizes the smooth approximation of the two-argument maximum function.

Instead of the nonsmooth

$$\max(a, b),$$

the code uses

$$m_\varepsilon(a, b) = \frac{1}{2}\left(a + b + \sqrt{(a-b)^2 + \varepsilon^2}\right).$$

Here `abs_eps_max` is that ε.

---

4. Smooth braking-slip magnitudes

```
% ===== Smooth braking-slip magnitudes from your SX definition =====
% Since SX is negative under braking, use smooth max(-SX, 0)
absSlip_fl = 0.5*((-SXfl) + sqrt(SXfl^2 + abs_eps_pos^2));
absSlip_fr = 0.5*((-SXfr) + sqrt(SXfr^2 + abs_eps_pos^2));
absSlip_rl = 0.5*((-SXrl) + sqrt(SXrl^2 + abs_eps_pos^2));
absSlip_rr = 0.5*((-SXrr) + sqrt(SXrr^2 + abs_eps_pos^2));
```

These four lines define four **regularized nonnegative braking-slip measures**, one per wheel.

Why `-SX` appears

Because your sign convention says:

- braking slip corresponds to `SX < 0`

the exact braking-slip magnitude of a wheel would naturally be written as

$$\max(-SX, 0).$$

That quantity is:

- positive when `SX < 0` (braking)
- zero when `SX >= 0` (non-braking or drive-side slip)

Why this formula is used

The code uses

$$\phi_\varepsilon(-SX) = \frac{1}{2}\left((-SX) + \sqrt{SX^2 + \varepsilon^2}\right)$$

which is a **smooth approximation** of $\max(-SX, 0)$.

For example,

MATLAB

is exactly

$$\text{absSlip}_{fl} = \frac{1}{2}\left(-SX_{fl} + \sqrt{SX_{fl}^2 + \varepsilon_{\text{pos}}^2}\right).$$

This is:

- smooth for all `SXfl`
- strictly positive for finite `abs_eps_pos`
- close to $\max(-SX_{fl}, 0)$ when `abs_eps_pos` is small

The same interpretation applies to:

- `absSlip_fr`
- `absSlip_rl`
- `absSlip_rr`

Important precise point

These are **not** exact braking-slip magnitudes.
They are **regularized smooth approximations** of the braking-slip magnitudes.

---

5. Smooth maxima over front axle, rear axle, and whole vehicle

```
% ===== Smooth maxima =====
absSlip_f = 0.5*(absSlip_fl + absSlip_fr + sqrt((absSlip_fl - absSlip_fr)^2 + abs_eps_max^2));
absSlip_r = 0.5*(absSlip_rl + absSlip_rr + sqrt((absSlip_rl - absSlip_rr)^2 + abs_eps_max^2));
```

MATLAB

These lines compute smooth approximations of maxima.

The generic smooth-max formula is:

$$m_\varepsilon(a, b) = \frac{1}{2}\left(a + b + \sqrt{(a-b)^2 + \varepsilon^2}\right).$$

For small ε, this is a smooth approximation of $\max(a, b)$.

Front axle quantity

MATLAB

This is

$$\text{absSlip}_f = m_{\varepsilon_{\max}}(\text{absSlip}_{fl}, \text{absSlip}_{fr}).$$

So `absSlip_f` is a smooth approximation of the larger of the two front-wheel braking-slip measures.

It is best interpreted as a **front-axle worst-wheel smooth braking-slip indicator**.

Rear axle quantity

MATLAB

This is

$$\text{absSlip}_r = m_{\varepsilon_{\max}}(\text{absSlip}_{rl}, \text{absSlip}_{rr}).$$

So `absSlip_r` is a smooth approximation of the larger of the two rear-wheel braking-slip measures.

It is a **rear-axle worst-wheel smooth braking-slip indicator**.

Global quantity

MATLAB

This is

$$\text{absSlip}_g = m_{\varepsilon_{\max}}(\text{absSlip}_f, \text{absSlip}_r).$$

So `absSlip_g` is a smooth approximation of the larger of:

- the front-axle worst-wheel indicator
- the rear-axle worst-wheel indicator

Hence it is a **global worst-wheel smooth braking-slip indicator**.

---

6. Smooth ABS gates

```
% ===== Smooth ABS gates =====
g_f   = 0.5*(1 + tanh((absSlip_f - abs_trig)/abs_width));
g_r   = 0.5*(1 + tanh((absSlip_r - abs_trig)/abs_width));
```

MATLAB

These lines map the smooth braking-slip indicators into smooth gate variables.

The generic formula is

$$g(x) = \frac{1}{2}\left(1 + \tanh\left(\frac{x - \text{abs\_trig}}{\text{abs\_width}}\right)\right).$$

For finite arguments, this satisfies

$$0 < g(x) < 1.$$

So each gate is a smooth scalar strictly between 0 and 1.

Front axle gate

MATLAB

This gate depends on the front-axle worst-wheel smooth slip indicator.

Interpretation:

- if `absSlip_f` is well below `abs_trig`, then `g_f` is close to 0
- if `absSlip_f` is well above `abs_trig`, then `g_f` is close to 1

So `g_f` is a **smooth front-axle ABS activation level**.

Rear axle gate

MATLAB

Same interpretation for the rear axle.

So `g_r` is a **smooth rear-axle ABS activation level**.

Global ABS gate

MATLAB

This depends on the global worst-wheel indicator.

So `g_abs` is a **smooth global ABS activation level** driven by the worst braking-slip condition on the vehicle, in a smooth approximate sense.

---

7. Global brake relief

```
% ===== Global brake relief while preserving exact front-rear ratio =====
% Tb <= 0, so multiplying by (1 - abs_eta*g_abs) makes braking less negative
```

MATLAB

This defines the effective brake command after smooth global ABS relief.

Exactly:

$$T_{b,\mathrm{eff}} = (1 - \eta\, g_{\mathrm{abs}})\, T_b$$

where:

- $\eta = \mathrm{abs\_eta}$
- $g_{\mathrm{abs}} \in (0, 1)$

Since `0 <= abs_eta < 1`, the multiplicative factor stays positive. Therefore:

- if `Tb <= 0`, then `Tb_eff <= 0`
- the sign of the brake command is preserved

Also, because the factor is less than or equal to 1,

$$|T_{b,\mathrm{eff}}| \le |T_b|.$$

So the magnitude of braking is not increased by this step.

Instead it is reduced smoothly as `g_abs` increases.

Why this preserves front-rear split later

This relief is applied to the **single total brake request** before front/rear splitting.
That is exactly why the axle-total front/rear ratio remains fixed afterward.

---

8. Exact axle-total front-rear brake split

```
% ===== Exact front-rear brake split =====
Tbf = kb * Tb_eff;          % total front brake torque <= 0
```

MATLAB

These lines define the effective front and rear axle brake torques.

Exactly:

$$T_{bf} = k_b\, T_{b,\mathrm{eff}}, \qquad T_{br} = (1 - k_b)\, T_{b,\mathrm{eff}}.$$

Properties

Sum property

$$T_{bf} + T_{br} = T_{b,\mathrm{eff}}.$$

So the total effective brake torque is exactly partitioned into front and rear axle totals.

Ratio property

If `Tb_eff ~= 0`, then

$$\frac{T_{bf}}{T_{br}} = \frac{k_b}{1 - k_b}.$$

So the front-rear split is exact at the axle-total level.

Sign property

Because `Tb_eff <= 0` and `0 < kb < 1`:

- `Tbf <= 0`
- `Tbr <= 0`

---

9. Smooth left-right redistribution within each axle

```
% ===== Smooth left-right redistribution inside each axle =====
% If left wheel has larger braking-slip magnitude than right,
% Delta > 0 and the left brake torque becomes less negative.
Delta_f = -0.5 * Tbf * g_f * tanh(abs_lr_gain * (absSlip_fl - absSlip_fr));
Delta_r = -0.5 * Tbr * g_r * tanh(abs_lr_gain * (absSlip_rl - absSlip_rr));
```

MATLAB

These lines create a smooth side-to-side brake-torque bias within each axle.

Front axle redistribution

$$\Delta_f = -\frac{1}{2}\, T_{bf}\, g_f\, \tanh\!\big(k_{lr}(\mathrm{absSlip}_{fl} - \mathrm{absSlip}_{fr})\big)$$

where $k_{lr} = \mathrm{abs\_lr\_gain}$.

Sign analysis

Because:

- `Tbf <= 0`
- `g_f > 0`
- `tanh(.)` lies in `(-1,1)`

it follows that `Delta_f` has the same sign as

$$\mathrm{absSlip}_{fl} - \mathrm{absSlip}_{fr}.$$

That means:

- if `absSlip_fl > absSlip_fr`, then `Delta_f > 0`
- if `absSlip_fl < absSlip_fr`, then `Delta_f < 0`
- if they are equal, then `Delta_f = 0`

Magnitude bound

Because $|\tanh(\cdot)| < 1$ for finite arguments and $0 < g_f < 1$,

$$|\Delta_f| < -\frac{1}{2} T_{bf}.$$

Since `Tbf <= 0`, the quantity $-\frac{1}{2}T_{bf}$ is nonnegative.

This bound matters because it guarantees redistribution does not exceed the half-axle braking magnitude.

Rear axle redistribution

Exactly the same structure:

$$\Delta_r = -\frac{1}{2}\, T_{br}\, g_r\, \tanh\!\big(k_{lr}(\mathrm{absSlip}_{rl} - \mathrm{absSlip}_{rr})\big).$$

So:

- `Delta_r > 0` when rear-left braking slip measure exceeds rear-right
- `Delta_r < 0` when rear-right exceeds rear-left

Again, the more-slipping wheel is the one that will later receive a less-negative brake torque.

Precise physical interpretation

These `Delta` terms do **not** change axle-total brake torque.
They only redistribute it between left and right wheels within the axle.

---

10. Front wheel brake torques

MATLAB

These lines assign the front axle brake torque to the two front wheels.

Exact decomposition

Without redistribution, each front wheel would get

$$\frac{1}{2}T_{bf}.$$

Redistribution adds:

- `+Delta_f` to the left
- `-Delta_f` to the right

So:

$$T_{bfl} = \frac{1}{2}T_{bf} + \Delta_f, \qquad T_{bfr} = \frac{1}{2}T_{bf} - \Delta_f.$$

Exact sum property

$$T_{bfl} + T_{bfr} = T_{bf}.$$

So front axle total brake torque is preserved exactly.

Exact sign property

Because $|\Delta_f| < -\frac{1}{2}T_{bf}$ for finite arguments, both

$$T_{bfl} \le 0, \qquad T_{bfr} \le 0$$

and in fact for finite values they remain strictly negative if `Tbf < 0`.

So redistribution cannot convert front brake torque into positive drive torque.

Slip-relief direction

If `absSlip_fl > absSlip_fr`, then `Delta_f > 0`, so

$$T_{bfl} = \frac{1}{2}T_{bf} + \Delta_f$$

becomes **less negative** than equal split, while

$$T_{bfr} = \frac{1}{2}T_{bf} - \Delta_f$$

becomes **more negative**.

So the more-slipping front wheel is relieved.

That statement is mathematically exact under the given sign convention.

---

11. Front total wheel torques

```
Tfl = Tbfl;
```

MATLAB

These define the final front wheel torques.

This means the front wheels have no applied drive torque in this formulation.
Their total applied torques are exactly their brake torques:

$$T_{fl} = T_{bfl}, \qquad T_{fr} = T_{bfr}.$$

So in the wheel dynamics,

- `Tfl` and `Tfr` are the torques that enter the front-wheel rotational equations.

---

12. Rear wheel brake torques

```
% ===== Rear wheel torques = equal drive split + rear brake split =====
Tbrl = 0.5*Tbr + Delta_r;
```

MATLAB

These are the rear-wheel brake torques, analogous to the front axle case.

Exactly:

$$T_{brl} = \frac{1}{2}T_{br} + \Delta_r, \qquad T_{brr} = \frac{1}{2}T_{br} - \Delta_r.$$

Sum property

$$T_{brl} + T_{brr} = T_{br}.$$

So rear axle total brake torque is preserved exactly.

Sign property

Using the same bound logic as above, both rear brake torques remain nonpositive:

$$T_{brl} \le 0, \qquad T_{brr} \le 0.$$

So the redistribution does not create positive rear brake torque.

Relief direction

If rear-left braking slip exceeds rear-right braking slip, then `Delta_r > 0`, so:

- `Tbrl` becomes less negative
- `Tbrr` becomes more negative

Thus the more-slipping rear wheel is relieved.

---

13. Final rear wheel torques

```
Trl = 0.5*Tt + Tbrl;
```

MATLAB

These lines define the final applied torques at the rear wheels.

Exactly:

$$T_{rl} = \frac{1}{2}T_t + T_{brl}, \qquad T_{rr} = \frac{1}{2}T_t + T_{brr}.$$

Exact meaning

The rear-wheel torque is the algebraic sum of:

- an equal split of the total rear drive request `Tt`
- the redistributed rear brake torque

So the code assumes:

- rear drive torque is distributed equally left/right
- rear braking may be unequal left/right due to ABS redistribution

---

Important sign interpretation

Because:

- `0.5*Tt >= 0`
- `Tbrl <= 0, Tbrr <= 0`

the final rear wheel torques may be:

- positive
- zero
- negative

depending on the balance between drive and brake contributions.

So this is an **algebraic superposition** of drive and brake torques at the rear wheels.

It does **not**, by itself, enforce a no-overlap supervisory rule such as "never command drive and brake simultaneously."

---

14. Exact overall structure of the block

The entire code does the following, mathematically:

Step A: build smooth per-wheel braking-slip indicators

$$\mathrm{absSlip}_{ij} \approx \max(-SX_{ij}, 0)$$

Step B: build smooth axle/global worst-slip indicators

$$\mathrm{absSlip}_f \approx \max(\text{front wheel slips}), \quad \mathrm{absSlip}_r \approx \max(\text{rear wheel slips}), \quad \mathrm{absSlip}_g \approx \max(\text{all wheel slips})$$

Step C: convert those into smooth activation gates

$$g_f,\ g_r,\ g_{\mathrm{abs}} \in (0, 1)$$

Step D: reduce the total brake request smoothly

$$T_{b,\mathrm{eff}} = (1 - \eta g_{\mathrm{abs}})T_b$$

Step E: split effective brake torque exactly front/rear

$$T_{bf} = k_b T_{b,\mathrm{eff}}, \qquad T_{br} = (1 - k_b)T_{b,\mathrm{eff}}$$

Step F: redistribute braking left/right inside each axle

$$\Delta_f, \Delta_r$$

so the more-slipping wheel is relieved

Step G: form final wheel torques

- front: braking only
- rear: equal drive split plus redistributed braking

---

15. Clean summary of what each line means

To make it maximally clear:

- `kb` sets the exact axle-total front brake fraction
- `abs_trig`, `abs_width` define where and how smoothly ABS activates
- `abs_eta` defines how much the total brake request can be reduced
- `abs_lr_gain` defines how strongly left-right brake redistribution reacts
- `abs_eps_pos` smooths `max(-SX,0)`
- `abs_eps_max` smooths `max(a,b)`

Then:

- `absSlip_fl`, `absSlip_fr`, `absSlip_rl`, `absSlip_rr` are smooth nonnegative braking-slip indicators
- `absSlip_f`, `absSlip_r`, `absSlip_g` are smooth worst-slip indicators
- `g_f`, `g_r`, `g_abs` are smooth activation levels
- `Tb_eff` is the ABS-relieved total brake command
- `Tbf`, `Tbr` are the exact axle-total front/rear brake torques
- `Delta_f`, `Delta_r` are the side-to-side redistribution amounts
- `Tbfl`, `Tbfr`, `Tbrl`, `Tbrr` are per-wheel brake torques
- `Tfl`, `Tfr`, `Trl`, `Trr` are the final applied wheel torques

---

16. One mathematically exact sentence for the whole block

This block defines a **smooth, fully differentiable rear-wheel-drive torque allocation law** in which braking slip is detected via regularized approximations of $\max(-SX, 0)$, the total brake request is reduced smoothly based on a global worst-wheel slip indicator, the axle-total front/rear brake split is preserved exactly, and within each axle the brake torque is redistributed so the wheel with larger braking-slip indicator receives less-negative brake torque.

If you want, I can next turn this into an even more useful format: your **original code pasted back with an exact explanation comment after every single line**.